

LangGraph Memory

Literal Code Execution: Checkpointers and State Persistence

In LangGraph, memory is implemented through checkpointers. A checkpointer persists the graph's state at every superstep, allowing the graph to pause, resume, or retain conversational history across multiple executions.

1. Initializing a Checkpointer

Memory requires a checkpointer object. LangGraph provides in-memory (`MemorySaver`) and persistent (e.g., SQLite, Postgres) checkpointers.

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver

# Literal execution: In-memory checkpointer
memory = MemorySaver()

# Literal execution: SQLite persistent checkpointer
# import sqlite3
# conn = sqlite3.connect("checkpoints.sqlite")
# memory = SqliteSaver(conn)
```

2. Compiling the Graph with Memory

To enable state saving, pass the checkpointer instance to the `checkpointer` argument when calling the `compile()` method on your `StateGraph`.

```
from langgraph.graph import StateGraph

# Assume workflow is a populated StateGraph(AgentState)
# workflow = StateGraph(AgentState)
# ... add nodes and edges ...

# Literal execution: Compile graph with checkpointer
app = workflow.compile(checkpointer=memory)
```

3. Executing with a Thread ID

When invoking a compiled graph with a checkpointer, you must provide a `thread_id` within a configurable dictionary. The `thread_id` maps the execution to a specific saved state.

```
# Define the execution payload
inputs = {"messages": [("user", "Configure the server for deployment.")]}

# Define the configuration with a unique thread_id
config = {"configurable": {"thread_id": "thread_123"}}

# Literal execution: Invoke graph passing the config
output = app.invoke(inputs, config=config)
```

Execution Flow: When `invoke` runs with a `thread_id`, the checkpointer queries the backend for existing state matching the ID. If found, it loads that state before processing the `inputs`. After execution, the updated state is written back.

4. Retrieving State Directly

The state for a specific thread can be queried directly from the compiled application without executing a new run.

```
# Literal execution: Fetch current state snapshot
snapshot = app.get_state(config)

# Access the state dictionary
current_state_values = snapshot.values
print(current_state_values["messages"])

# Literal execution: Fetch history of states for a thread
state_history = list(app.get_state_history(config))
```